Firewall Linux, Arquitetura Netfilter e iptables

Aluna: Iasmyn Severo
Repositório: https://github.com/myhticile/firewallsi

1. Definição do Case

O cenário escolhido é uma loja virtual fictícia (MurkTech), que vende eletrônicos. ela é baseada em como pequenas lojas online funcionam de verdade.
Na MurkTech, os funcionários de atendimento e vendas usam computadores na rede interna da empresa. eles precisam acessar o site da loja, que fica em um servidor web. os pedidos e os dados dos clientes ficam guardados em um banco de dados.
Os serviços que precisam funcionar são:

Serviço	Tecnologia	Para que serve	Quem deve acessar
Servidor web	Nginx	Site e painel da loja	Funcionários
Banco de dados	PostgreSqL	Guardar pedidos e cadastro de clientes	Só a aplicação, nunca os computadores dos funcionários
Estação de trabalho	Alpine Linux	Computador de atendimento e vendas	Acessa o site, mas não o banco

2. Diagnóstico: cenário atual vs. proposta de solução

Cenário atual (problemas mapeados)

Hoje a MurkTech usa uma rede plana: os computadores dos funcionários, o servidor web e o banco de dados estão todos juntos, sem nenhum controle entre eles. causando os seguintes problemas:

- **Gargalos de tráfego:** como tudo está na mesma rede, o tráfego dos funcionários, do site e do banco se mistura. Uma varredura de portas ou uma transferência pesada em um ponto afeta todos os outros, e não existe como limitar ou separar o que circula.
- **Falta de segmentação:** qualquer computador da rede consegue acessar o banco de dados direto, pela porta 5432.
- **Ausência de controle de acesso:** alguém mal-intencionado, ou um computador infectado, pode varrer as portas de todos os servidores com ferramentas como o nmap.
- **Risco de a invasão se espalhar:** se um computador for invadido, o atacante chega fácil ao banco. Se o servidor web for invadido, ele chega aos computadores dos funcionários.

- **Serviços expostos sem necessidade:** portas como a do SSH ficam abertas para toda a rede.
- **Nenhum registro:** ninguém fica sabendo quando alguém tenta acessar algo que não devia.
- **Risco para os dados:** o banco guarda dados pessoais e não tem nenhuma barreira de proteção.

Proposta de solução

A proposta é separar a rede em duas zonas e colocar um firewall no meio, controlando tudo que passa de uma zona para a outra. o firewall usa o iptables (do Netfilter, o sistema de filtragem de pacotes do Linux) com uma regra básica: tudo é bloqueado por padrão, e só passa o que foi liberado de propósito.

As duas zonas, simuladas com Docker Compose, são:

Zona	Rede	Faixa de IP	O que tem dentro
Cliente	cliente_net	172.20.0.0/24	Computador de atendimento (seginf-cliente, 172.20.0.10)
DMZ	dmz_net	172.21.0.0/24	Servidor web (seginf-web, 172.21.0.10) e banco de dados (seginf-banco, 172.21.0.20)

O firewall (seginf-firewall) está nas duas redes ao mesmo tempo (172.20.0.2 e 172.21.0.2). Por isso, ele é o único caminho entre elas. Ele roda com a permissão `NET_ADMIN`, necessária para mexer nas regras, e com o encaminhamento de pacotes ligado (`net.ipv4.ip_forward=1`), para funcionar como roteador. As regras são carregadas automaticamente pelo script `entrypoint.sh` quando o container liga. O banco de dados usa um volume do Docker, para que os dados não sejam perdidos se o container for recriado.

Por que essa arquitetura:

- **Por que uma DMZ:** os servidores ficam separados dos computadores dos funcionários. Qualquer acesso a eles precisa passar pelo firewall.
- **Por que bloquear tudo por padrão:** se alguém criar um serviço novo e esquecer de proteger, ele já nasce bloqueado.
- **Por que um firewall que acompanha as conexões (stateful):** ele só deixa passar as respostas de conversas que já foram autorizadas.
- **Por que bloquear o banco com registro em log:** o cliente não precisa falar com o banco, e qualquer tentativa fica anotada.

- **Por que usar NAT (MASQUERADE):** o servidor web e o banco não sabem o caminho de volta para a rede do cliente. Então o firewall troca o endereço de origem pelo dele mesmo, e as respostas voltam por ele.

3. Matriz de Regras de Segurança (Firewall)

A política padrão do firewall é `INPUT DROP`, `FORWARD DROP` e `OUTPUT ACCEPT`. As regras estão no arquivo `firewall/entrypoint.sh`.

Origem	Destino	Porta/Protocolo	Ação	Justificativa do negócio
Rede Cliente	DMZ (Web)	TCP 80 e 443	ACCEPT	Funcionários precisam acessar o sistema da loja
Rede Cliente	DMZ (Web)	ICMP (ping)	ACCEPT	Testar se o servidor está no ar
Rede Cliente	DMZ (DB)	TCP 5432	LOG + DROP	O cliente não deve acessar o banco direto, e a tentativa fica registrada
Qualquer	Qualquer	Respostas de conexões já liberadas	ACCEPT	Permitir que as respostas das conexões autorizadas voltem
Qualquer	Firewall	Qualquer (exceto loopback e respostas)	DROP	O firewall não oferece serviço nenhum
Qualquer	Qualquer	Todo o resto	DROP	Se não foi liberado de propósito, é bloqueado (inclui ping ao banco e SSH ao servidor web)
Rede Cliente	Rede DMZ	Todos	MASQUERADE (NAT)	O servidor web e o banco não têm rota de volta para a rede do cliente

4. Metodologia de Testes e Validação

Passo a passo para subir o ambiente

-Abra o Docker Desktop e abra um terminal
-Baixe o projeto e entre na pasta: git clone https://github.com/myhticile/firewallsi.git
cd firewallsi
- Suba o ambiente: docker compose up -d --build

- Espere alguns segundos. O container do cliente instala as ferramentas (curl, nmap, netcat)
- quando liga. Quando o comando abaixo mostrar a mensagem cliente pronto, comece os testes: docker logs --tail 5 seginf-cliente
- Para ver as regras carregadas no firewall: docker logs --tail 25 seginf-firewall
- Dentro do cliente, os nomes dmz-web e dmz-db apontam para o servidor web e para o banco.

Testes positivos (o que deve funcionar)

Teste	Comando	O que deve acontecer
P1: acessar o site	`docker exec seginf-cliente curl -s http://dmz-web`	Aparece a página “Welcome to nginx!”
P2: pingar o servidor web	`docker exec seginf-cliente ping -c 2 dmz-web`	0% de perda de pacotes
P3: verificar portas do servidor web	`docker exec seginf-cliente nmap -Pn -p 80,443 dmz-web`	Porta 80 aberta e porta 443 fechada

A porta 443 aparece como fechada porque o firewall deixa o tráfego passar, mas o Nginx só escuta na porta 80.

Testes negativos (o que deve ser bloqueado)

Teste	Comando	O que deve acontecer
N1: conectar no banco	`docker exec seginf-cliente nc -zv -w 3 dmz-db 5432`	Dá timeout (não conecta)
N2: nmap no banco	`docker exec seginf-cliente nmap -Pn -p 5432 dmz-db`	Porta aparece como `filtered`
N3: pingar o banco	`docker exec seginf-cliente ping -c 2 -W 2 dmz-db`	100% de perda de pacotes
N4: acessar SSH do servidor web	`docker exec seginf-cliente nc -zv -w 3 dmz-web 22`	Dá timeout (bloqueado pela política padrão)

Como provar que foi o firewall que bloqueou

Depois dos testes, rode o comando abaixo. A coluna pkts mostra números maiores que zero nas regras de liberar e de bloquear: docker exec seginf-firewall iptables -L FORWARD -n -v

Resultados obtidos

P1: o comando devolveu o HTML da página "Welcome to nginx!".

P2: o ping funcionou. O ttl=63 (e não 64) mostra que o pacote passou por um roteador no caminho, que é o firewall. 64 bytes from dmz-web (172.21.0.10): icmp_seq=1 ttl=63 time=0.295 ms 64 bytes from dmz-web (172.21.0.10): icmp_seq=2 ttl=63 time=0.110 ms 2 packets transmitted, 2 received, 0% packet loss

P3: PORT STATE SERVICE 80/tcp open http 443/tcp closed https

N1: nc: connect to dmz-db (172.21.0.20) port 5432 (tcp) timed out

N2: PORT STATE SERVICE 5432/tcp filtered postgresql

N3: 2 packets transmitted, 0 received, 100% packet loss

N4: nc: connect to dmz-web (172.21.0.10) port 22 (tcp) timed out

Contadores do firewall depois dos testes:
Chain FORWARD (policy DROP 5 packets, 348 bytes)

pkts	bytes	target	prot	source	destination	
20	2285	ACCEPT	0	0.0.0.0/0	0.0.0.0/0	ctstate RELATED,ESTABLISHED
5	236	ACCEPT	6	172.20.0.10	172.21.0.10	multiport dports 80,443
1	84	ACCEPT	1	172.20.0.10	172.21.0.10	icmptype 8
5	268	LOG	6	172.20.0.10	172.21.0.20	tcp dpt:5432
5	268	DROP	6	172.20.0.10	172.21.0.20	tcp dpt:5432

Como ler esse resultado: 5 pacotes do cliente tentando chegar ao banco foram registrados e bloqueados pela regra específica. Outros 5 pacotes (o ping ao banco e a tentativa de SSH no servidor web) foram bloqueados pela política padrão, porque nenhuma regra os liberava.

Para encerrar: docker compose down